

Contents

Tutorial completo — Claude Code + SendFlow (SendAPI) via MCP	1
Sumário	1
1. O que você vai construir	1
2. Pré-requisitos	2
3. Parte 1 — Instalar as ferramentas base	2
4. Parte 2 — Criar a estrutura do projeto	3
5. Parte 3 — Configurar o ambiente Python	4
6. Parte 4 — Registrar o MCP no Claude Code	5
7. Parte 5 — Testar a integração	5
8. Parte 6 — Uso no dia a dia	6
9. Troubleshooting	6
10. Anexo A — Conteúdo completo dos arquivos	8
11. Checklist final (para confirmar que está tudo certo)	9
12. Créditos e evolução	9

Tutorial completo — Claude Code + SendFlow (SendAPI) via MCP

Guia passo a passo para integrar o **Claude Code** (rodando no VS Code) com a **SendAPI da SendFlow** através de um servidor **MCP customizado em Python**, sem depender de Make, Zapier ou outra ferramenta intermediária.

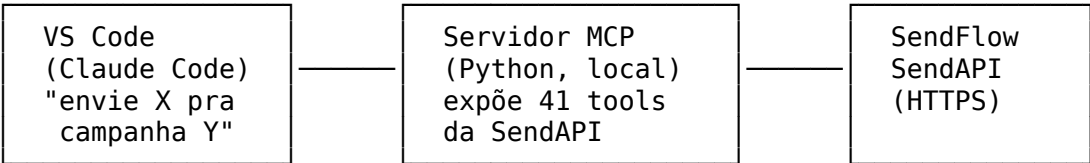
Ao final deste tutorial, você poderá falar com o Claude em linguagem natural dentro do VS Code e ele chama a SendAPI diretamente — disparar mensagens, listar campanhas, criar grupos, verificar números, etc.

Este documento foi escrito focando em **Windows 10/11** (o cenário que foi usado para validar o tutorial). Onde houver diferença no macOS/Linux, há uma nota específica.

Sumário

- 1. [O que você vai construir](#)
- 2. [Pré-requisitos](#)
- 3. [Parte 1 — Instalar as ferramentas base](#)
- 4. [Parte 2 — Criar a estrutura do projeto](#)
- 5. [Parte 3 — Configurar o ambiente Python](#)
- 6. [Parte 4 — Registrar o MCP no Claude Code](#)
- 7. [Parte 5 — Testar a integração](#)
- 8. [Parte 6 — Uso no dia a dia](#)
- 9. [Troubleshooting](#)
- 10. [Anexo A — Conteúdo completo dos arquivos](#)

1. O que você vai construir



- **Claude Code** é o cliente: você conversa com ele em linguagem natural dentro do terminal do VS Code.
- **Servidor MCP** (Model Context Protocol) é um processo Python local que traduz os pedidos do Claude em chamadas HTTP na SendAPI.
- **SendAPI** é a API REST da SendFlow (<https://sendflow.pro/sendapi>).

Resultado: quando você pede algo como “mande ‘teste’ para a campanha X”, o Claude Code chama a tool correta (`send_text_to_campaign`), o servidor MCP monta a requisição autenticada e devolve a resposta da SendFlow.

2. Pré-requisitos

- Uma **conta SendFlow** com plano que permita uso da API.
 - Uma **API Key da SendFlow** (formato `send_api-...`). Se ainda não tiver, gere em: <https://dubble.so/guides/sendia-criando-o-token-api-vgujvjqox8vatfjhhemw>
 - Um PC com **Windows 10 ou 11** (ou macOS/Linux — este tutorial foca em Windows).
 - Conexão com a internet.
-

3. Parte 1 — Instalar as ferramentas base

Você precisa de **5 ferramentas** instaladas antes de começar.

3.1 Python 3.10 ou superior

Baixe em <https://www.python.org/downloads/> e instale. **Muito importante:** na primeira tela do instalador, marque a caixa **“Add Python to PATH”** antes de clicar em Install.

Depois de instalar, abra um terminal (PowerShell) e confirme:

```
python --version
```

Precisa retornar Python 3.10.x ou superior (3.11, 3.12, etc.).

No macOS/Linux o comando é geralmente `python3 --version`.

3.2 Node.js (LTS)

O Claude Code é distribuído como pacote npm, então precisa do Node.js.

Baixe a versão **LTS** em <https://nodejs.org/en/download> e instale com as opções padrão.

Depois confirme:

```
node --version
```

```
npm --version
```

Ambos precisam retornar números de versão.

3.3 Git for Windows (com Git Bash) — obrigatório no Windows

O Claude Code, no Windows, precisa do `git-bash.exe` para funcionar.

Baixe em <https://git-scm.com/downloads/win> e instale com as opções padrão (apenas clique “Next” em todas as telas).

Depois de instalado, o `bash.exe` deve existir em:

```
C:\Program Files\Git\bin\bash.exe
```

Definir a variável de ambiente CLAUDE_CODE_GIT_BASH_PATH:

Abra o PowerShell **como administrador** (menu Iniciar → clique com o direito em “Windows PowerShell” → “Executar como administrador”) e rode:

```
[System.Environment]::SetEnvironmentVariable('CLAUDE_CODE_GIT_BASH_PATH', 'C:\Program Files\Claude Code')
```

Feche esse PowerShell. Em qualquer novo terminal, você pode confirmar com:

```
echo $env:CLAUDE_CODE_GIT_BASH_PATH
```

3.4 VS Code

Baixe em <https://code.visualstudio.com/Download> e instale.

3.5 Claude Code

Com Node.js já instalado, abra o **PowerShell** (qualquer um, não precisa ser admin) e rode:

```
npm install -g @anthropic-ai/claude-code
```

A instalação leva de 1 a 2 minutos. No final, confirme:

```
claude --version
```

Se o claude não for reconhecido, o PATH do npm global não está no seu ambiente. Veja [seção de troubleshooting](#).

Faça login com sua conta:

```
claude login
```

Um navegador vai abrir para autenticar. Depois de confirmar, volte ao terminal.

4. Parte 2 — Criar a estrutura do projeto

O projeto é uma pasta com 7 arquivos. Você pode:

- **Opção A:** copiar a pasta sendflow-mcp que já está pronta nesse repo.
- **Opção B:** criar os arquivos um por um seguindo o conteúdo do [Anexo A](#).

4.1 Criar a pasta

Escolha um lugar no seu computador (ex: Documentos) e crie:

```
cd C:\Users\SeuNome\Documents
mkdir sendflow-mcp
cd sendflow-mcp
```

4.2 Arquivos a criar

Dentro dessa pasta, você precisa ter exatamente estes arquivos:

sendflow-mcp/	
├── server.py	# o servidor MCP em Python (~500 linhas)
├── requirements.txt	# dependências Python
├── .env.example	# template da variável de ambiente
├── .mcp.example.json	# exemplo de config do Claude Code
├── claude_desktop_config.example.json	# exemplo de config do Claude Desktop
├── .gitignore	# padrões ignorados pelo Git
└── README.md	# doc resumida

O conteúdo de cada arquivo está no [Anexo A](#). Se você estiver copiando da pasta pronta, simplesmente duplique a pasta inteira.

5. Parte 3 — Configurar o ambiente Python

Aqui você cria um **ambiente virtual (venv)** — um Python isolado só para esse projeto, para não poluir seu sistema.

5.1 Abrir a pasta no VS Code

No VS Code, menu **File → Open Folder...** → selecione a pasta `sendflow-mcp`.

Se perguntar se você confia nos autores, clique em **Yes, I trust the authors**.

5.2 Abrir o terminal integrado

Atalho: `Ctrl + `` (Ctrl + crase, acima da tecla Tab).

Confirme que está na pasta certa:

```
pwd
```

Deve mostrar algo como `C:\Users\SeuNome\Documents\sendflow-mcp`.

5.3 Criar e ativar o venv

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Se aparecer erro de **execution policy**, rode isso uma vez só:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Confirme com S e tente ativar de novo. Quando funcionar, o prompt passa a mostrar `(.venv)` no começo da linha.

No macOS/Linux o comando de ativação é `source .venv/bin/activate`.

5.4 Instalar as dependências

Com o venv ativo:

```
pip install -r requirements.txt
```

Isso instala mcp, httpx e python-dotenv. Deve terminar sem erros vermelhos.

5.5 Criar o arquivo .env com a API key

```
Copy-Item .env.example .env
```

No macOS/Linux: `cp .env.example .env`.

No VS Code, clique no arquivo `.env` no explorer lateral e substitua o valor pela sua API key real:

```
SENDFLOW_API_KEY=send_api-coloque_sua_chave_real_aqui
```

Salve com `Ctrl + S`.

5.6 Testar que o servidor sobe

```
python server.py
```

Se o terminal **ficar parado sem imprimir nada**, é sinal de que está tudo certo — o servidor MCP fica aguardando conexão via stdio.

Aperte Ctrl + C para encerrar.

6. Parte 4 — Registrar o MCP no Claude Code

Agora você diz ao Claude Code que esse servidor existe e como iniciá-lo.

Com o terminal na pasta sendflow-mcp:

```
claude mcp add sendflow --env SENDFLOW_API_KEY=send_api-SUA_KEY -- "$PWD\.venv\Scripts\python
```

Substitua send_api-SUA_KEY pela sua API key real.

No macOS/Linux:

```
claude mcp add sendflow --env SENDFLOW_API_KEY=send_api-SUA_KEY -- "$(pwd)/.venv/bin/pyth
```

Confirme que foi adicionado:

```
claude mcp list
```

Deve mostrar sendflow na lista.

Escopo do MCP:

Por padrão o `claude mcp add` usa escopo **local** (só funciona dentro dessa pasta). Se quiser que o sendflow esteja disponível em qualquer pasta do seu computador, use `--scope user` e caminhos absolutos:

```
claude mcp remove sendflow
```

```
claude mcp add sendflow --scope user --env SENDFLOW_API_KEY=send_api-SUA_KEY -- "C:\Users\Seu
```

7. Parte 5 — Testar a integração

No terminal do VS Code, dentro da pasta do projeto:

```
claude
```

Isso abre a interface do Claude Code dentro do terminal.

7.1 Primeiro teste — leitura (não dispara nada)

Dentro do Claude Code, digite:

Usando a ferramenta sendflow, liste minhas contas de WhatsApp.

Ele vai pedir autorização para usar a tool `list_accounts`. Aprove com y. Se voltar um JSON com suas contas, **a integração está funcionando**.

7.2 Segundo teste — envio real

Use a ferramenta sendflow para enviar uma mensagem de texto “Teste Claude Code + SendAPI” para a campanha de releaseId RELEASE_ID, usando o accountId ACCOUNT_ID.

Substitua RELEASE_ID e ACCOUNT_ID pelos valores reais (pegue com o primeiro teste + list_campaigns). Ao aprovar, o Claude chama send_text_to_campaign, a SendAPI responde {"message": "Ação criada com sucesso", "id": "..."} e em alguns segundos a mensagem aparece nos grupos da campanha.

8. Parte 6 — Uso no dia a dia

A partir daqui é tudo linguagem natural. Exemplos úteis:

- **Monitoramento:** *"Me mostra o analytics das minhas últimas 3 campanhas e qual teve mais cliques hoje."*
- **Envio agendado:** *"Agende uma mensagem com texto 'Promoção relâmpago' para a campanha X às 18h de hoje com shippingSpeed normal."*
- **Limpeza de base:** *"Verifique se os números 11999999999, 11888888888 e 11777777777 estão bloqueados para a campanha X."*
- **Bloqueio rápido:** *"Bloqueie o número 5511999999999 como 'Spammer' e confirme."*
- **Gestão de contas:** *"Me mostra o QR code da conta ACC123 para eu reconectar o WhatsApp."*
- **Criação:** *"Crie uma campanha 'Black Friday 2026' e depois adiciona o grupo 120363292004848696@g.us nela."*

O Claude Code escolhe a tool certa das 41 disponíveis e chama a SendAPI. Se precisar de dados que não existem nas tools (ex: filtros locais, formatação para planilha), ele combina com leitura/escrita de arquivo.

9. Troubleshooting

Esses são os erros mais comuns com base em casos reais.

9.1 source: 0 termo 'source' não é reconhecido (Windows)

Causa: source é comando Unix/Mac. No PowerShell do Windows, ative o venv com:

```
.venv\Scripts\Activate.ps1
```

9.2 execution of scripts is disabled on this system

Causa: política de execução do PowerShell bloqueia o script de ativação.

Solução: libere scripts locais (uma vez só):

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Confirme com S e tente ativar de novo.

9.3 claude não é reconhecido

Causa 1: o Claude Code não foi instalado. Rode `npm install -g @anthropic-ai/claude-code`.

Causa 2: instalado, mas o PATH do npm global não está no ambiente. Descubra onde o npm instalou:

```
npm config get prefix
```

Retorna algo como `C:\Users\SeuNome\AppData\Roaming\npm`.

Para **usar imediatamente**, chame pelo caminho completo:

```
& "C:\Users\SeuNome\AppData\Roaming\npm\claude.cmd" --version
```

Para **corrigir permanentemente**, adicione esse caminho ao PATH:

1. Tecla Windows → “variáveis de ambiente” → **Editar as variáveis de ambiente do sistema**
2. Botão **Variáveis de Ambiente...**
3. Em “Variáveis do usuário”, selecione **Path** → **Editar...**
4. Clique **Novo** e cole o caminho (ex: C:\Users\SeuNome\AppData\Roaming\npm)
5. OK em todas as janelas
6. Feche o VS Code inteiro e reabra.

9.4 Claude Code on Windows requires git-bash

Causa: o Git for Windows não está instalado ou a variável CLAUDE_CODE_GIT_BASH_PATH não está definida.

Solução: instale o Git em <https://git-scm.com/downloads/win> e depois, em um PowerShell admin:

```
[System.Environment]::SetEnvironmentVariable('CLAUDE_CODE_GIT_BASH_PATH', 'C:\Program Files\Git\bin')
```

Feche o VS Code e reabra.

9.5 MCP server sendflow already exists in local config

Causa: você já tinha registrado o sendflow antes.

Solução: se quer substituir:

```
claude mcp remove sendflow
```

```
claude mcp add sendflow --env SENDFLOW_API_KEY=... -- "$PWD\.venv\Scripts\python.exe" "$PWD\server.py"
```

Ou simplesmente prossiga — a mensagem é informativa, não é erro.

9.6 SENDFLOW_API_KEY não está definida

Causa: o servidor subiu mas não achou a key.

Solução:

- Confirme que o .env existe na pasta e contém SENDFLOW_API_KEY=send_api-...
- Ou confirme que o --env SENDFLOW_API_KEY=... foi passado no claude mcp add.

A key do --env tem precedência sobre o .env.

9.7 401 Unauthorized nas chamadas da SendAPI

Causa: API key inválida, expirada ou de um usuário sem permissão.

Solução: gere uma nova key no SendFlow, atualize .env e re-registre o MCP.

9.8 Claude Code não lista as tools do sendflow

Causa: o server.py não está subindo (erro de caminho, Python do venv não encontrado, etc.).

Solução: rode manualmente para ver o erro:

```
"$PWD\.venv\Scripts\python.exe" "$PWD\server.py"
```

Se der erro, corrija. Se rodar sem erro (fica parado esperando), o caminho está certo — remova/re-adicione o MCP com exatamente o mesmo caminho.

10. Anexo A — Conteúdo completo dos arquivos

Se você não tiver a pasta pronta, use este anexo para recriar cada arquivo do zero. Todos precisam estar dentro da pasta `sendflow-mcp/`.

10.1 requirements.txt

```
mcp>=1.2.0
httpx>=0.27.0
python-dotenv>=1.0.0
```

10.2 .env.example

```
# Copie este arquivo para `.env` e preencha com seus valores reais.
SENDFLOW_API_KEY=send_api-coloque_sua_chave_aqui
# SENDFLOW_BASE_URL=https://sendflow.pro
# SENDFLOW_TIMEOUT=60
```

10.3 .gitignore

```
.env
.venv/
venv/
__pycache__/
*.pyc
*.pyo
.mypy_cache/
.ruff_cache/
.pytest_cache/
```

10.4 .mcp.example.json

```
{
  "mcpServers": {
    "sendflow": {
      "command": "/CAMINHO/ABSOLUTO/sendflow-mcp/.venv/bin/python",
      "args": [
        "/CAMINHO/ABSOLUTO/sendflow-mcp/server.py"
      ],
      "env": {
        "SENDFLOW_API_KEY": "send_api-coloque_sua_chave_aqui"
      }
    }
  }
}
```

No Windows, use caminhos estilo `C:\Users\...\sendflow-mcp\.venv\Scripts\python.exe`.

10.5 claude_desktop_config.example.json

Mesmo conteúdo do `.mcp.example.json` — use-o como modelo para o arquivo `claude_desktop_config.json` do Claude Desktop (em macOS: `~/Library/Application Support/Claude/claude_desktop_config.json`; em Windows: `%APPDATA%\Claude\claude_desktop_config.json`).

10.6 server.py

O código completo do servidor MCP (com as 41 tools) está no arquivo `server.py` que acompanha este tutorial na pasta `sendflow-mcp`. São cerca de 500 linhas de Python organizadas em 8 seções que espelham a documentação da SendAPI:

1. Campanhas (`/releases`) — 10 tools
2. Grupos de campanhas (`/sendapi/release-groups`) — 4 tools
3. Ações (`/sendapi/actions`) — envio em grupos — 9 tools
4. Mensagens diretas (`/sendapi/send-*/{accountId}`) — 4 tools
5. Templates (`/sendapi/message-templates`) — 4 tools
6. Contas (`/sendapi/accounts`) — 7 tools
7. Números bloqueados (`/sendapi/block-numbers`) — 2 tools
8. Verificação de número (`/sendapi/verify-number`) — 1 tool

Para recriar do zero, o arquivo está disponível nesta pasta — basta copiar. Se quiser expandir com mais tools (ex: workflows customizados, envio em lote com validação prévia de bloqueio), é só adicionar mais funções decoradas com `@mcp.tool()` e reiniciar o Claude Code.

10.7 README.md

Versão curta do tutorial, focada em uso rápido. Está na pasta `sendflow-mcp`.

11. Checklist final (para confirmar que está tudo certo)

- ☐ `python --version` retorna 3.10 ou superior
- ☐ `node --version` e `npm --version` retornam versões
- ☐ Git for Windows instalado e `echo $env:CLAUDE_CODE_GIT_BASH_PATH` retorna o caminho
- ☐ VS Code aberto na pasta `sendflow-mcp`
- ☐ `.venv` criado e ativado (prompt mostra `(.venv)`)
- ☐ `pip install -r requirements.txt` rodou sem erro
- ☐ Arquivo `.env` existe com a `SENDFLOW_API_KEY` correta
- ☐ `python server.py` fica parado aguardando (e não imprime erro)
- ☐ `claude --version` responde com um número
- ☐ `claude mcp list` mostra `sendflow`
- ☐ Dentro do `claude`, o prompt “liste minhas contas de WhatsApp” retorna um JSON da SendAPI

Se todos os itens estão marcados, sua integração Claude + VS Code + SendAPI está pronta. Bom uso!

12. Créditos e evolução

- A SendAPI oficial: <https://sendflow.pro/sendapi>
- Claude Code: <https://docs.anthropic.com/en/docs/claude-code>
- Protocolo MCP: <https://modelcontextprotocol.io>

Contribuições, sugestões e extensões do `server.py` são bem-vindas. Para adicionar novas tools, copie o padrão das existentes em `server.py` e reinicie o Claude Code.